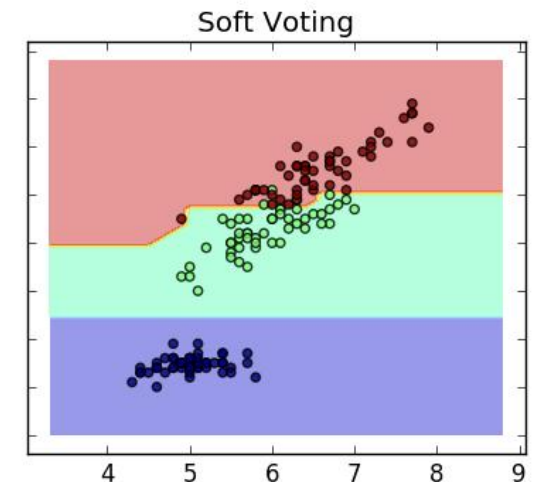
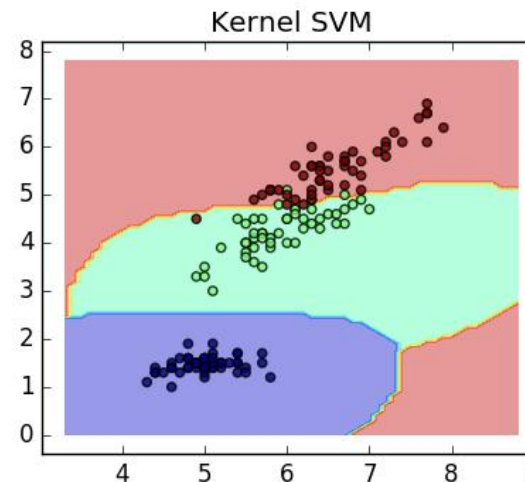
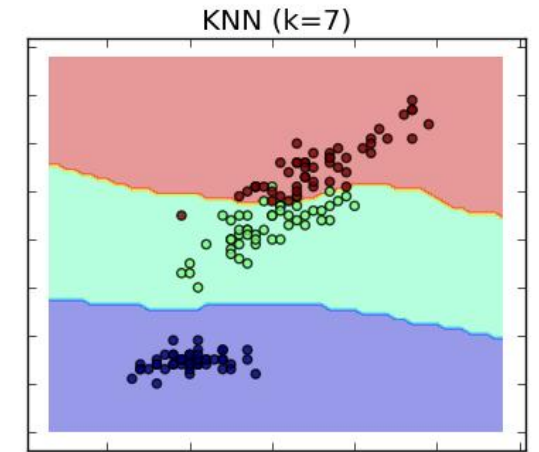
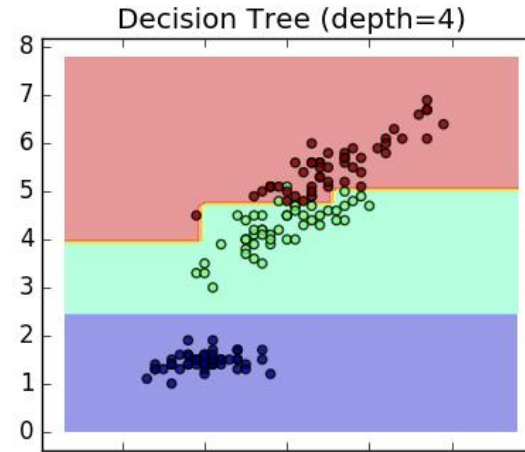


Ensembles

Voting

- **Hard voting**
uses predicted class labels for majority rule voting
- **Soft voting**
predicts the class label based on the argmax of the sums of the predicted probabilities



Ensemble Learning

Let $\{Y_i\}_{i=1}^n$ is a set of uncorrelated random variables with common mean μ and variance σ^2 .

$$\mathbb{E} \left[\frac{1}{n} \sum_{i=1}^n Y_i \right] = \frac{1}{n} \cdot n\mu = \mu$$

$$\text{Var} \left(\frac{1}{n} \sum_{i=1}^n Y_i \right) = \left(\frac{1}{n} \right)^2 \sum_{i=1}^n \text{Var}(Y_i) = \frac{1}{n^2} n\sigma^2 = \frac{\sigma^2}{n}$$

Random Forest

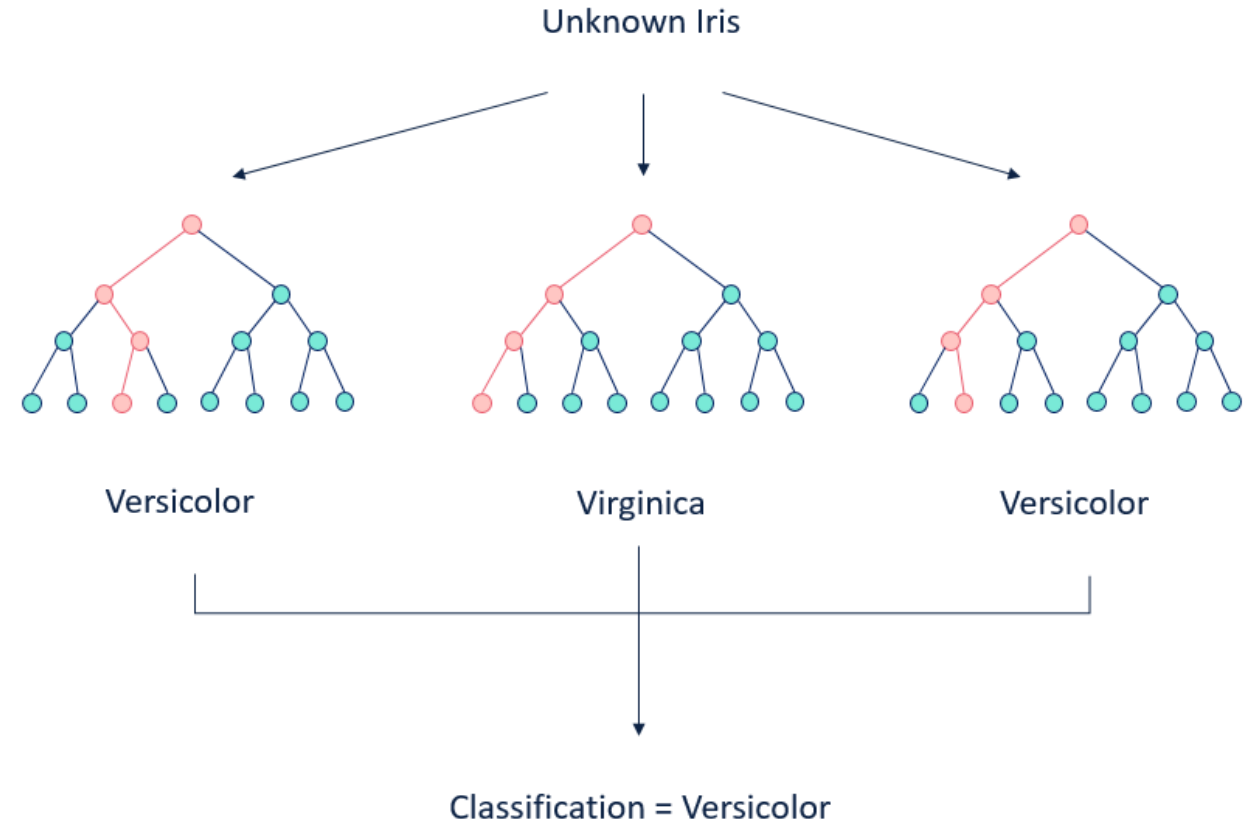
Train many small trees on random subsamples.

Sampling types:

- **Pasting:** simple sampling with no repeats
- **Bagging** (bootstrap aggregating): sample with repeats of the same size as the original dataset
- **Random Subspaces:** sample of features
- **Random Patches:** sample both features and examples

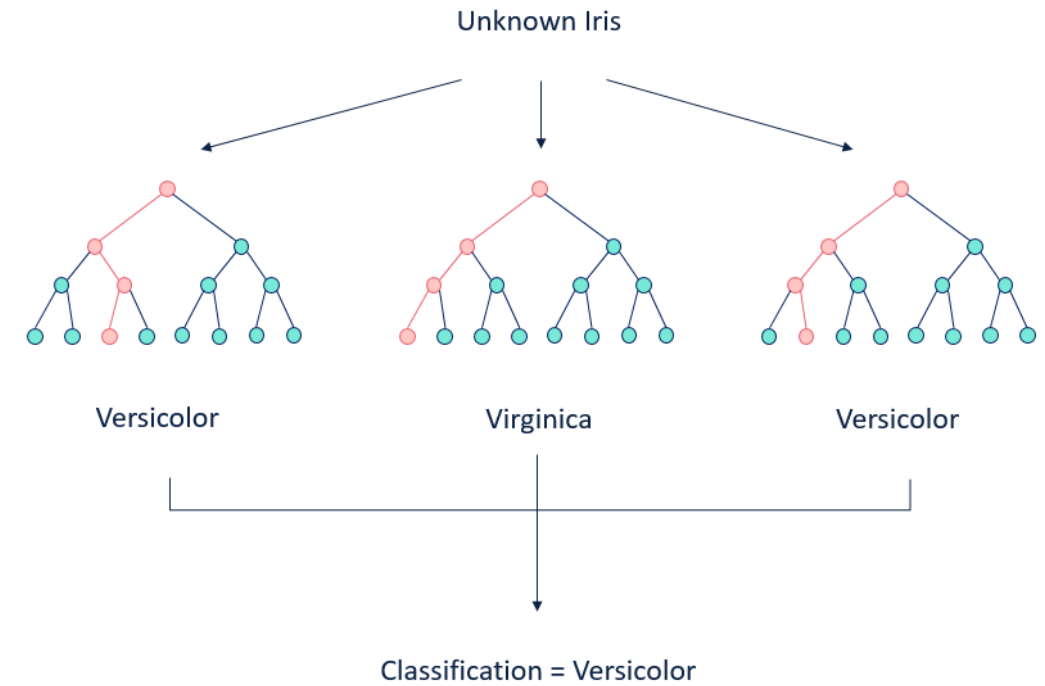
Final result could be determined by either hard or soft voting.

You can even use extremely randomized trees!

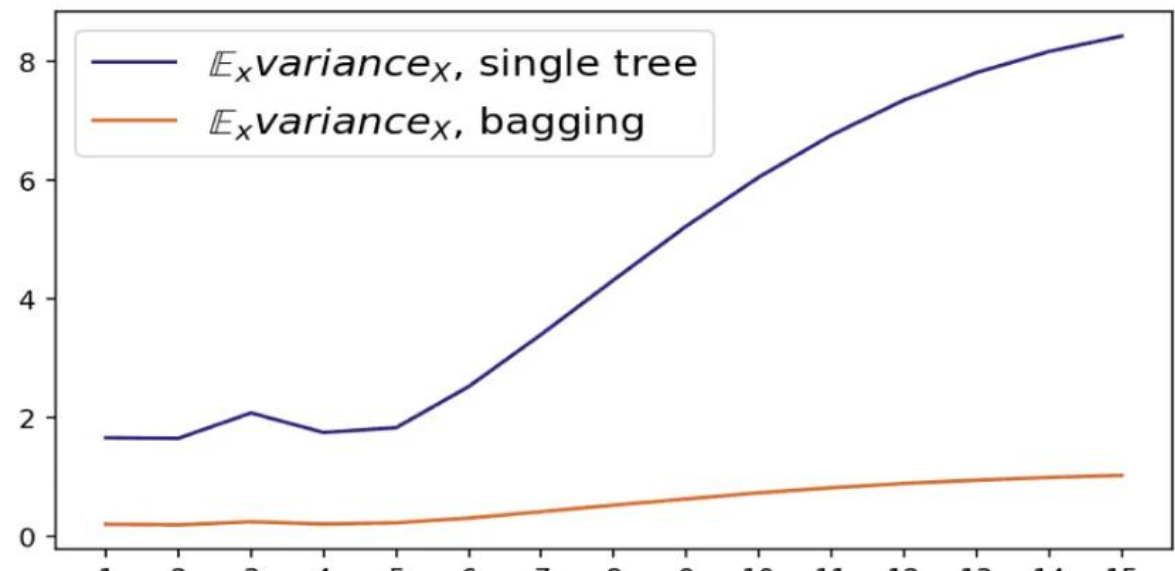
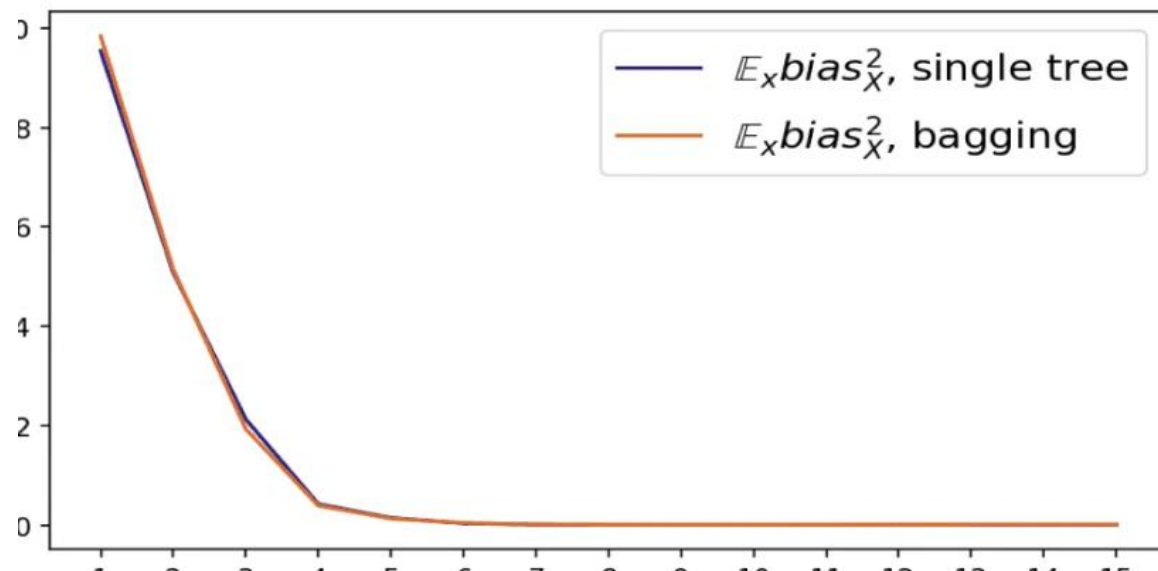
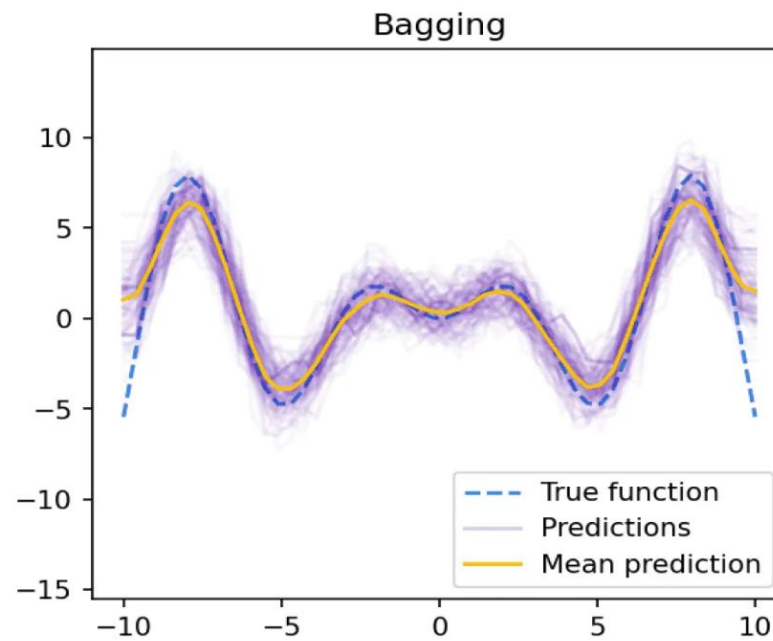
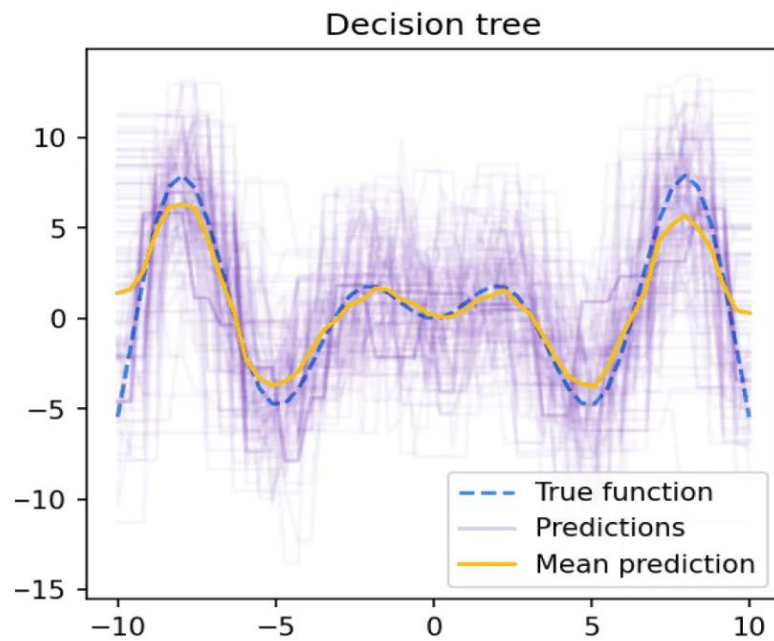


Random Forest: Algorithm

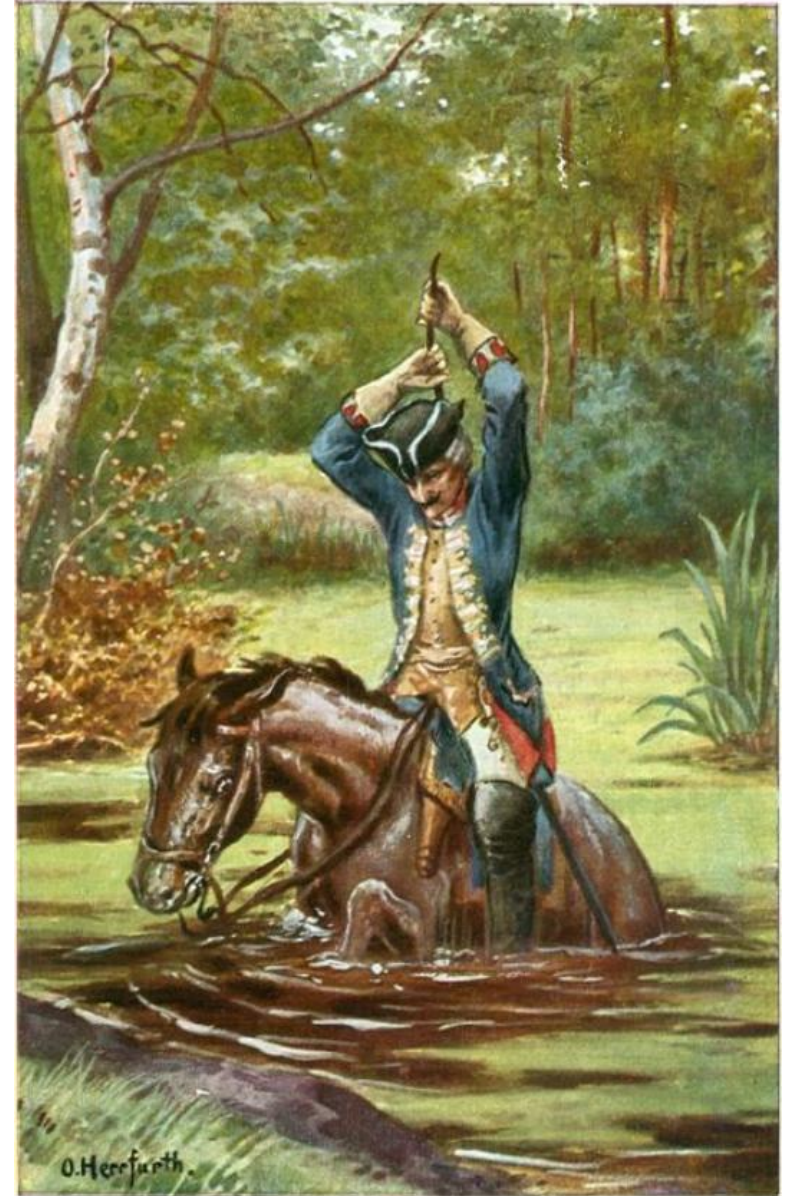
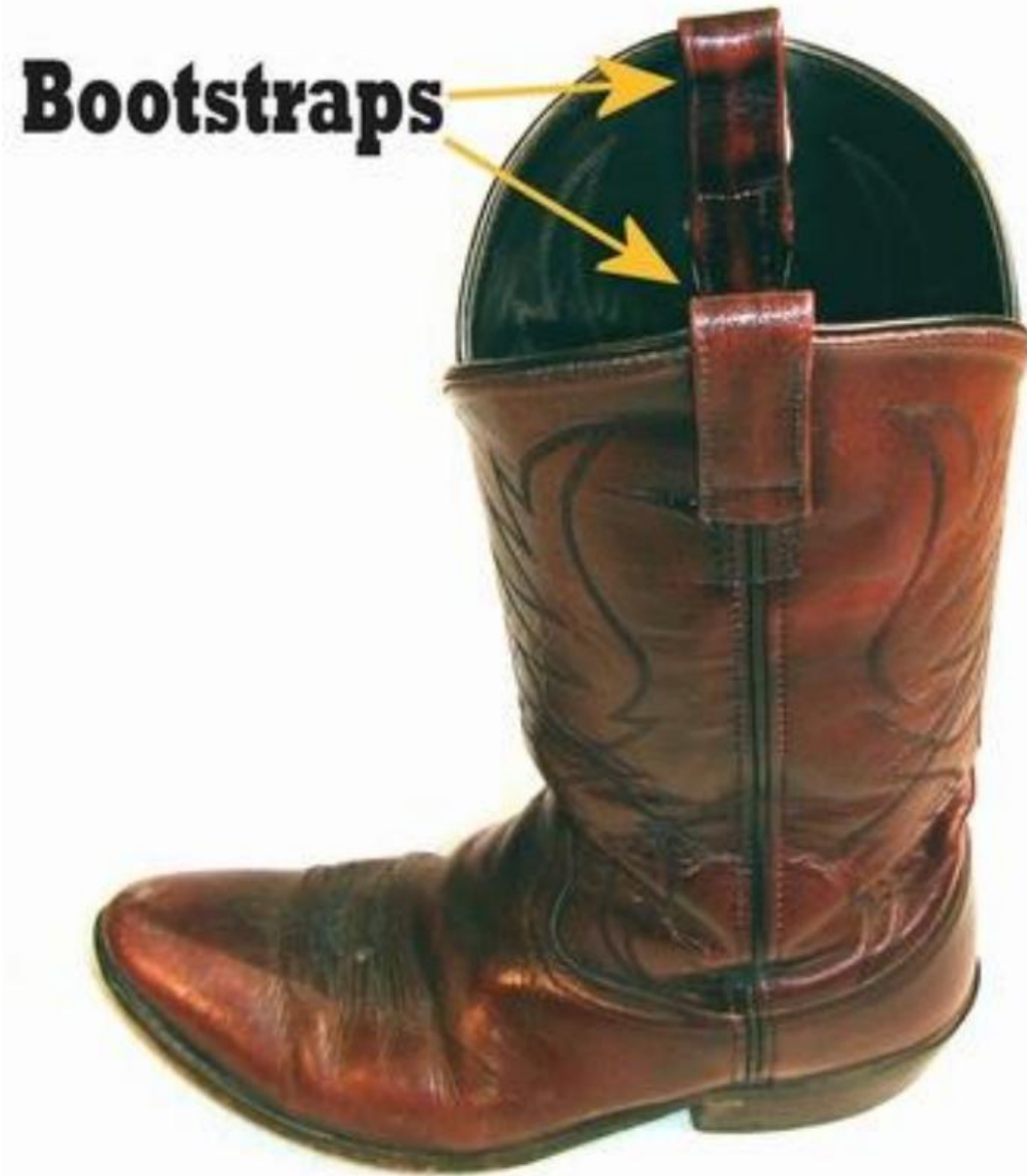
- Use bagging to train each tree
- Use a sample of features at each node in your tree
- Answer is average across all trees (regression) or voting (classification)



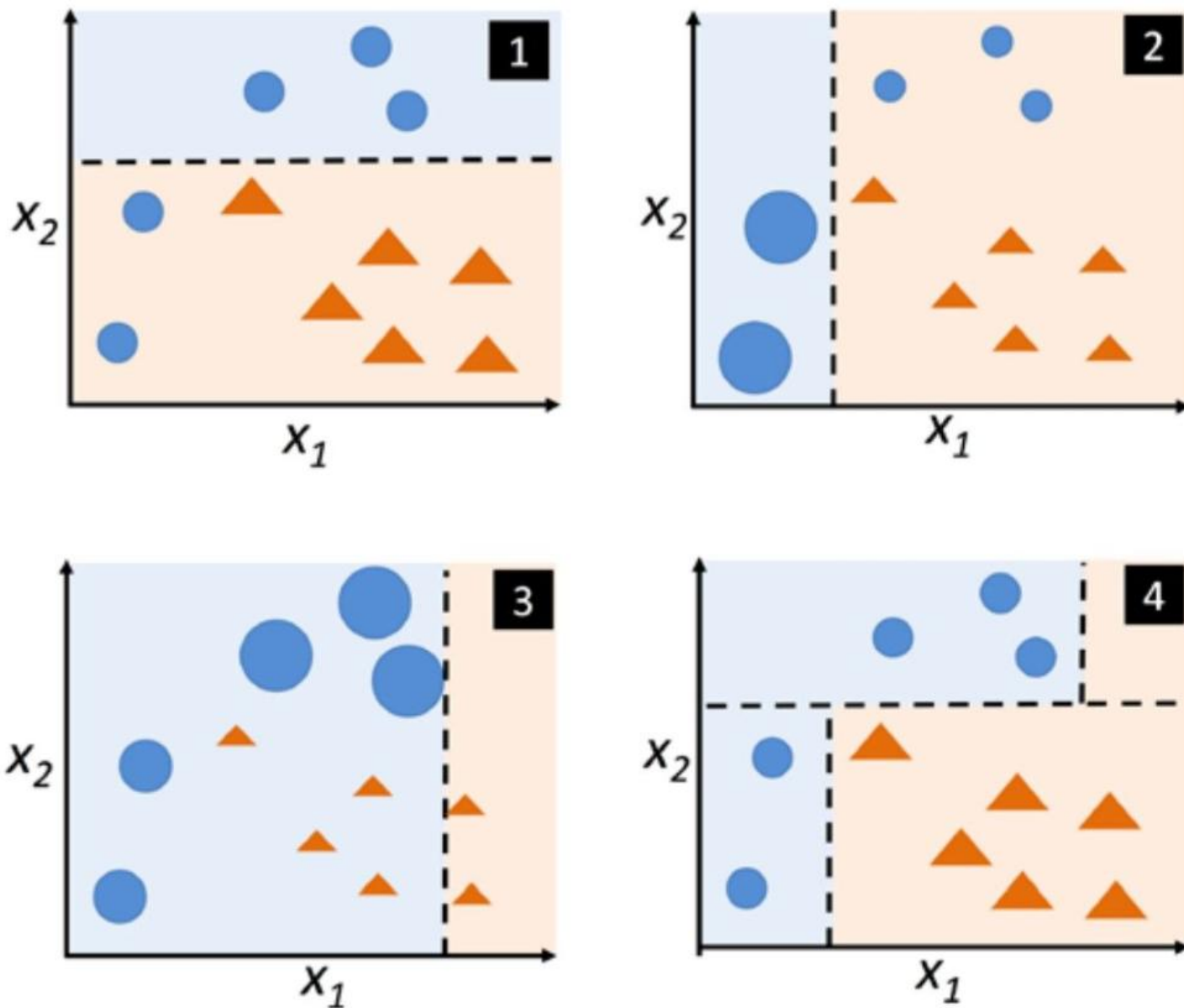
Random Forest



Pulling yourself up by your bootstraps



Adaptive Boosting (AdaBoost)



Adaptive Boosting (AdaBoost)

- Data: $\{(x_1, y_1), \dots, (x_N, y_N)\}, y_i \in \{-1, 1\}$
- Set of weak classifiers: $\{k_1, \dots, k_L\}: k_j(x_i) \in \{-1, 1\}$

At $(m - 1)$ -th iteration: $C_{m-1}(x_i) = \alpha_1 k_1(x_1) + \dots + \alpha_{m-1} k_{m-1}(x_i)$

At m -th iteration: $C_m(x_i) = C_{m-1}(x_i) + \alpha_m k_m(x_i)$

How we can determine α_m and k_m ?

Total error of C_m – sum of exponential loss on each data point:

$$E = \sum_{i=1}^N e^{-y_i C_m(x_i)} = \sum_{i=1}^N e^{-y_i C_{m-1}(x_i)} e^{-y_i \alpha_m k_m(x_i)} = \sum_{i=1}^N w_i^m e^{-y_i \alpha_m k_m(x_i)}$$

where $w_i^1 = 1, w_i^m = e^{-y_i C_{m-1}(x_i)}$ for $m > 1$

AdaBoost: Selecting a Classifier

$$\begin{aligned} E &= \sum_{i=1}^N w_i^m e^{-y_i \alpha_m k_m(x_i)} = \sum_{y_i = k_m(x_i)} w_i^m e^{-\alpha_m} + \sum_{y_i \neq k_m(x_i)} w_i^m e^{\alpha_m} \\ &= \sum_{i=1}^N w_i^m e^{-\alpha_m} + \underbrace{\sum_{y_i \neq k_m(x_i)} w_i^m (e^{\alpha_m} - e^{-\alpha_m})}_{\text{Depends on } k_m} \end{aligned}$$

➤ Our next classifier has the lowest weighted error $\sum_{y_i \neq k_m(x_i)} w_i^m$

AdaBoost: Weight Calculation

How we can find α_m ?

$$\frac{dE}{d\alpha_m} = \frac{d\left(\sum_{y_i=k_m(x_i)} w_i^m e^{-\alpha_m} + \sum_{y_i \neq k_m(x_i)} w_i^m e^{\alpha_m}\right)}{d\alpha_m}$$

$$\frac{dE}{d\alpha_m} = - \sum_{y_i=k_m(x_i)} w_i^m e^{-\alpha_m} + \sum_{y_i \neq k_m(x_i)} w_i^m e^{\alpha_m} = 0$$

$$e^{-\alpha_m} \sum_{y_i=k_m(x_i)} w_i^m = e^{\alpha_m} \sum_{y_i \neq k_m(x_i)} w_i^m \Rightarrow \alpha_m = \frac{1}{2} \ln \left(\frac{\sum_{y_i=k_m(x_i)} w_i^m}{\sum_{y_i \neq k_m(x_i)} w_i^m} \right)$$

$$\text{Let } \epsilon_m = \frac{\sum_{y_i \neq k_m(x_i)} w_i^m}{\sum_{i=1}^N w_i^m}, \text{ then } \alpha_m = \frac{1}{2} \ln \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$$

AdaBoost: Algorithm

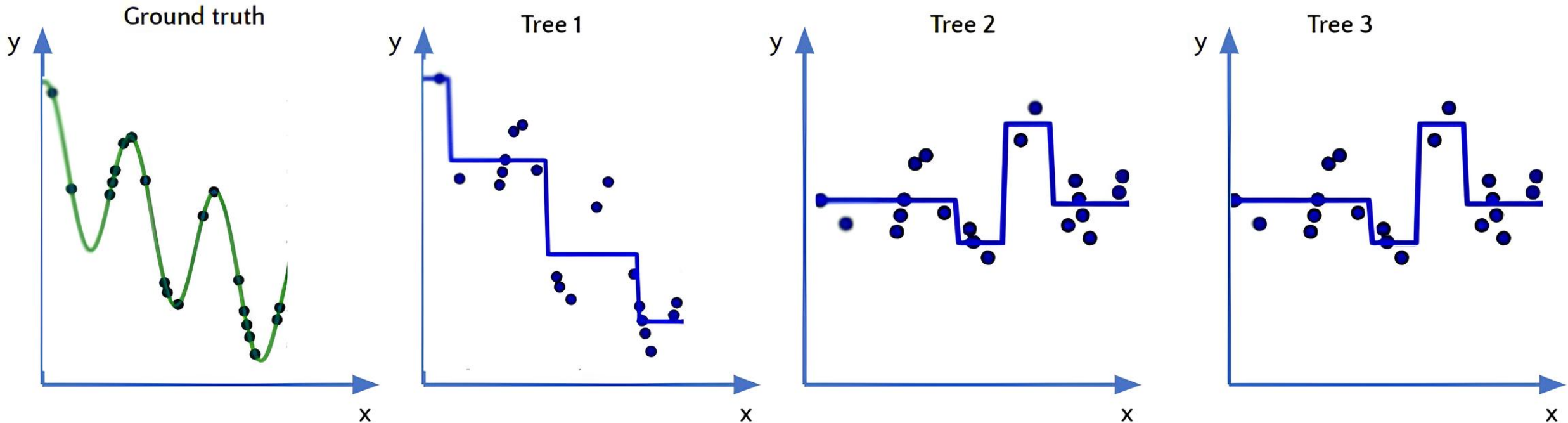
At each iteration:

- Choose the classifier k_m which minimizes $\sum_{y_i \neq k_m(x_i)} w_i^m$
- Then calculate $\epsilon_m = \frac{\sum_{y_i \neq k_m(x_i)} w_i^m}{\sum_{i=1}^N w_i^m}$
- By ϵ_m calculate $\alpha_m = \frac{1}{2} \ln \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$
- Improve the boosted classifier C_{m-1} to $C_m = C_{m-1} + \alpha_m k_m$

Gradient Boosting

$$H_{t+1}(x) = H_t + h_{t+1}(x) \rightarrow y \Rightarrow h_{t+1}(x) \rightarrow y - H_t(x)$$

$$h_{t+1}(x) \rightarrow \underbrace{y - H_t(x)}_{\text{residual}}$$



Informal View

We have: $\{(x_1, y_1), \dots, (x_N, y_N)\}$

We need: teach model F to predict y_i by minimizing MSE

Our model: $F_{m+1}(x_i) = F_m(x_i) + h_m(x_i) = y_i \Leftrightarrow h_m(x_i) = y_i - F_m(x_i)$

Our target for current step: $L_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - F(x_i))^2$

$$\underbrace{-\frac{\partial L_{MSE}}{\partial F(x_i)}}_{\text{antigradient}} = \frac{2}{n} (y_i - F(x_i)) = \frac{2}{n} h_m(x_i)$$

More Formally

We have: $\{(x_i, y_i)\}_{i=1}^N$ – data, $L(y, F(x))$ – loss, M – number of iterations

Algorithm:

1. Initialize model (constant value): $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$
2. On step m :
 1. Compute residuals (pseudo-residuals): $r_i^m = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$
 2. Fit a weak learner h_m closed to residuals
 3. Update the model: $F_m(x) = F_{m-1}(x) + \eta h_m(x)$

Extreme Gradient Boosting (XGBoost)

We have: $\{(x_i, y_i)\}_{i=1}^N$ – data, $L(y, F(x))$ – loss

We want: $\sum_{i=1}^N L(y_i, F_{m-1}(x_i) + h(x_i)) \rightarrow \min_h$

$$\sum_{i=1}^N L(y_i, F_{m-1}(x_i) + h(x_i)) \approx \sum_{i=1}^N \left(L(y_i, F_{m-1}(x_i)) - s_i h(x_i) + \frac{1}{2} v_i h^2(x_i) \right)$$

$$s_i = -\frac{\partial L(y, F_{m-1}(x))}{\partial F_{m-1}(x_i)}, \quad v_i = \frac{\partial^2 L(y, F_{m-1}(x))}{\partial F_{m-1}^2(x_i)}$$

$$\sum_{i=1}^N \left(-s_i h(x_i) + \frac{1}{2} v_i h^2(x_i) \right) \rightarrow \min_h$$

Back to MSE

$$\sum_{i=1}^N (h(x_i) - s_i)^2 = \sum_{i=1}^N (h^2(x_i) - 2s_i h(x_i) + s_i^2) = \{s_i^2 \text{ doesn't depend on } h\}$$

$$= \sum_{i=1}^N (h^2(x_i) - 2s_i h(x_i)) = 2 \sum_{i=1}^N \left(-s_i h(x_i) + \frac{1}{2} h^2(x_i) \right) \rightarrow \min_h$$

XGBoost: Regularization

A tree can be represented as: $h(x) = \sum_{j=1}^J h_j[x \in R_j]$

The complexity of the tree depends on:

1. leaf numbers: J
2. norms of coefficients in leaves: $\|h\|_2^2 = \sum_{j=1}^J h_j^2$

Final problem:

$$\sum_{i=1}^N \left(-s_i h(x_i) + \frac{1}{2} v_i h^2(x_i) \right) + \gamma J + \frac{\lambda}{2} \sum_{j=1}^J h_j^2 \rightarrow \min_h$$

XGBoost

$$\sum_{i=1}^N \left(-s_i h(x_i) + \frac{1}{2} v_i h^2(x_i) \right) + \gamma J + \frac{\lambda}{2} \sum_{j=1}^J h_j^2 \rightarrow \min_h$$

The tree gives the same answers to objects that fall on the same leaf:

$$\sum_{j=1}^J \left\{ \left(-\sum_{i \in R_j} s_i \right) h_j + \frac{1}{2} \left(\lambda + \sum_{i \in R_j} v_i \right) h_j^2 + \gamma \right\} \rightarrow \min_h$$

$$\sum_{j=1}^J \left\{ -S_j h_j + \frac{1}{2} (\lambda + V_j) h_j^2 + \gamma \right\} \rightarrow \min_h$$

For a fixed structure $h(x)$: $h_j = \frac{S_j}{V_j + \lambda} \Rightarrow L(h) = -\frac{1}{2} \sum_{j=1}^J \frac{S_j^2}{V_j + \lambda} + \gamma J$

XGBoost

$$L(h) = -\frac{1}{2} \sum_{j=1}^J \frac{s_j^2}{v_j + \lambda} + \gamma J$$

For a fixed structure we can use it like the impurity score:

$$Q = L(R) - L(R_l) - L(R_r) \rightarrow \max$$

In practice:

$$L_{split} = \frac{1}{2} \left[\frac{(\sum_{i \in R_l} s_i)^2}{\sum_{i \in R_l} v_i + \lambda} + \frac{(\sum_{i \in R_r} s_i)^2}{\sum_{i \in R_r} v_i + \lambda} - \frac{(\sum_{i \in R} s_i)^2}{\sum_{i \in R} v_i + \lambda} \right] - \gamma$$